

WEEK-7

1. Implement the following optimization algorithms on deep neural networks

a) Stochastic Gradient Descent b) Ada Grad c) Adam

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.datasets import mnist
from tensorflow.keras.optimizers import SGD, Adagrad, Adam
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = x_train.reshape(-1, 28*28) / 255.0
```

```
x_test = x_test.reshape(-1, 28*28) / 255.0
```

```
y_train = to_categorical(y_train, 10)
```

```
y_test = to_categorical(y_test, 10)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 2s 0us/step

```
def create_model():
    model = keras.Sequential([
        keras.layers.Dense(128, activation='relu', input_shape=(784,)),
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dense(10, activation='softmax')
    ])
    return model
```

```
epochs = 10
```

```
batch_size = 1000
```

```
model = create_model()
```

```
sgd_optimizer = SGD(learning_rate=0.01)
```

```
model.compile(optimizer=sgd_optimizer,
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

```
sgd_history = model.fit(x_train, y_train,
                       epochs=epochs,
                       batch_size=batch_size,
                       validation_data=(x_test, y_test))
```

```
model = create_model()
```

```
adagrad_optimizer = Adagrad(learning_rate=0.01)
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

(AUTONOMOUS)

DEPARTMENT OF CSE(AI&ML)

```
model= create_model()
adagrad_optimizer= Adagrad(learning_rate=0.01)
model.compile(optimizer=adagrad_optimizer,
              loss='categorical_crossentropy',
              metrics=['accuracy'])

adagrad_history= model.fit(x_train, y_train,
                          epochs=epochs,
                          batch_size=batch_size,
                          validation_data=(x_test, y_test))

model=create_model()

adam_optimizer=Adam(learning_rate=0.001)
model.compile(optimizer=adam_optimizer,
              loss='categorical_crossentropy',
              metrics=['accuracy'])

adam_history=model.fit(x_train, y_train,
                      epochs=epochs,
                      batch_size=batch_size,
                      validation_data=(x_test, y_test))
```

```
plt.plot(sgd_history.history['accuracy'], label="SGD")
plt.plot(adagrad_history.history['accuracy'], label="Adagrad")
plt.plot(adam_history.history['accuracy'], label="Adam")
plt.title("Accuracy comparision of optimizers")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT

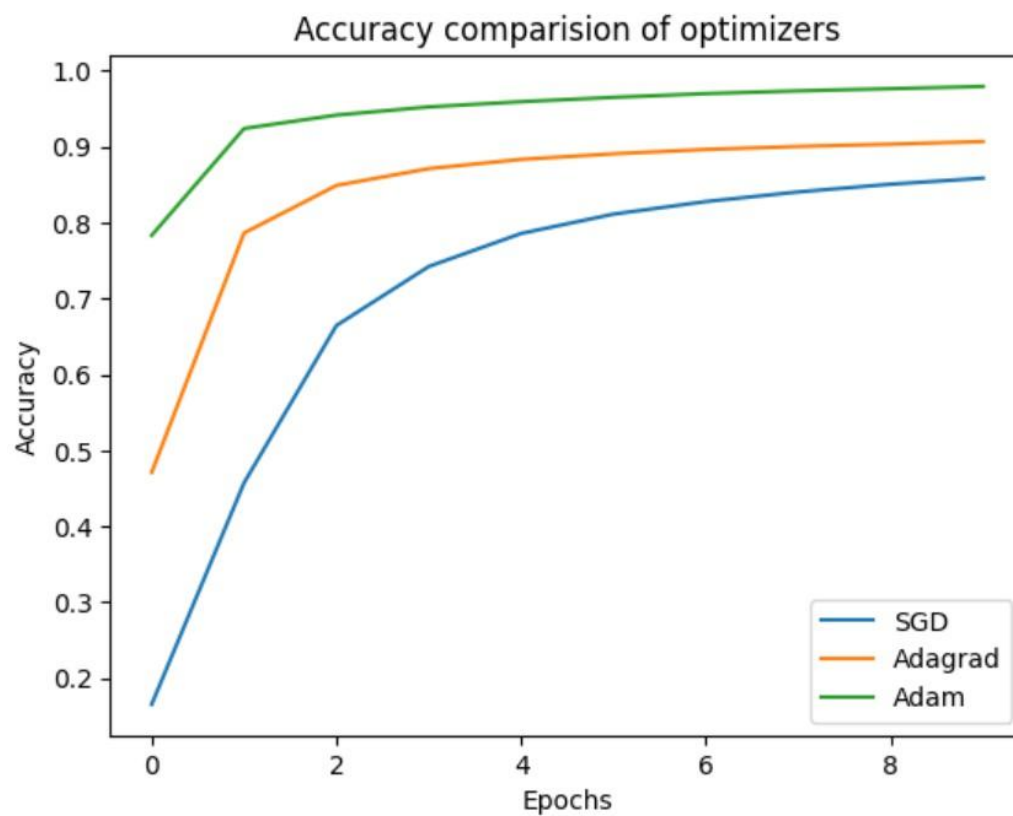
```
Epoch 1/10
60/60 ————— 3s 33ms/step - accuracy: 0.1107 - loss: 2.3074 - val_accuracy: 0.3112 - val_loss: 2.066
6
Epoch 2/10
60/60 ————— 1s 21ms/step - accuracy: 0.3858 - loss: 1.9881 - val_accuracy: 0.6003 - val_loss: 1.706
4
Epoch 3/10
60/60 ————— 1s 21ms/step - accuracy: 0.6336 - loss: 1.6242 - val_accuracy: 0.7173 - val_loss: 1.339
1
Epoch 4/10
60/60 ————— 1s 20ms/step - accuracy: 0.7281 - loss: 1.2795 - val_accuracy: 0.7821 - val_loss: 1.051
6
Epoch 5/10
60/60 ————— 1s 20ms/step - accuracy: 0.7786 - loss: 1.0165 - val_accuracy: 0.8126 - val_loss: 0.862
8
Epoch 6/10
60/60 ————— 1s 21ms/step - accuracy: 0.8052 - loss: 0.8532 - val_accuracy: 0.8321 - val_loss: 0.741
8
Epoch 7/10
60/60 ————— 1s 20ms/step - accuracy: 0.8231 - loss: 0.7407 - val_accuracy: 0.8475 - val_loss: 0.659
5
Epoch 8/10
60/60 ————— 3s 20ms/step - accuracy: 0.8376 - loss: 0.6650 - val_accuracy: 0.8566 - val_loss: 0.600
1
Epoch 9/10
60/60 ————— 1s 20ms/step - accuracy: 0.8466 - loss: 0.6106 - val_accuracy: 0.8657 - val_loss: 0.555
3
```

23251A6651

Sreshta

CSM – A

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE
(AUTONOMOUS)
DEPARTMENT OF CSE(AI&ML)



WEEK-8

1. Write a program to implement Convolution neural network on any image dataset.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
from tensorflow.keras.regularizers import l1, l2
(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()
train_images.shape
```

(60000, 28, 28)

```
train_images = train_images.reshape((60000, 28, 28, 1))
test_images = test_images.reshape((10000, 28, 28, 1))
train_images, test_images = train_images/255.0, test_images/255.0
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu', kernel_regularizer=l1(0.01), activity_regularizer=l2(0.01)),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = model.fit(train_images, train_labels, epochs=5, validation_data=(test_images, test_labels))
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1875/1875 — 81s 43ms/step - accuracy: 0.8315 - loss: 2.4559 - val_accuracy: 0.9607 - val_loss: 0.1859
Epoch 2/5
1875/1875 — 80s 43ms/step - accuracy: 0.9597 - loss: 0.1866 - val_accuracy: 0.9704 - val_loss: 0.1336
Epoch 3/5
1875/1875 — 80s 43ms/step - accuracy: 0.9668 - loss: 0.1390 - val_accuracy: 0.9759 - val_loss: 0.0983
Epoch 4/5
1875/1875 — 80s 43ms/step - accuracy: 0.9733 - loss: 0.1073 - val_accuracy: 0.9797 - val_loss: 0.0943
Epoch 5/5
1875/1875 — 80s 42ms/step - accuracy: 0.9758 - loss: 0.0962 - val_accuracy: 0.9793 - val_loss: 0.0802
```

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test accuracy: {test_acc}")
```

```
313/313 — 4s 12ms/step - accuracy: 0.9749 - loss: 0.0911
Test accuracy: 0.9793000221252441
```

```
train_loss, train_acc = model.evaluate(train_images, train_labels)
print(f"Train accuracy: {train_acc}")
```

```
1875/1875 — 22s 12ms/step - accuracy: 0.9790 - loss: 0.0824
Train accuracy: 0.9786166548728943
```

```
train_labels.shape
```

(60000,)

23251A6651

Sreshta

CSM – A

WEEK-9

1. Write a program to find the similarity between word documents using neural networks.

```
import tensorflow as tf

import tensorflow_hub as hub

module_url="https://tfhub.dev/google/universal-sentence-encoder/4"

embed=hub.load(module_url)

def calculate_similarity(doc1,doc2):

    embeddings=embed([doc1,doc2])

    print("embedding:",embeddings)

    similarity=tf.keras.losses.cosine_similarity(embeddings[0],embeddings[1])

    return similarity

def print_similarity_score(doc1,doc2):

    similarity_score=calculate_similarity(doc1,doc2)

    print("similarity score:",similarity_score)

doc1="A fast brown fox leaps over a sleepy canine."

doc2="A fast brown fox leaps over a sleepy canine."

print_similarity_score(doc1,doc2)
```

OUTPUT

```
... embedding: tf.Tensor(
  [[-0.04734416 -0.00069751  0.02375235 ... -0.00875956 -0.0381063
    0.02760308]
  [-0.04734416 -0.00069751  0.02375235 ... -0.00875956 -0.0381063
    0.02760308]], shape=(2, 512), dtype=float32)
similarity score: tf.Tensor(-1.0, shape=(), dtype=float32)
```

WEEK-10

1. Write a program to build speech to text model using deep neural networks.

```
!pip install speechrecognition pydub

import speech_recognition as sr
import requests
from pydub import AudioSegment

def add_punctuation(text):
    url = "http://bark.phon.ioc.ee/punctuator"
    response = requests.post(url, data={"text": text})
    return response.text

# File paths
mp3_file = "/content/Standard recording 1.mp3"
wav_file = "/content/Standard recording 1.wav"

# Convert MP3 to WAV
AudioSegment.from_mp3(mp3_file).export(wav_file, format="wav")

recognizer = sr.Recognizer()

with sr.AudioFile(wav_file) as source:
    audio = recognizer.record(source)

try:
    text = recognizer.recognize_google(audio)
    print(add_punctuation(text))
except Exception as e:
    print("Error:", e)
```

[How can I ins](#)

OUTPUT

```
... Requirement already satisfied: speechrecognition in /usr/local/lib/python3.12/dist-packages (3.14.5)
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (0.25.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from speechrecognition) (4.15.0)
My name is Siri vennela
```

WEEK-11

1.Case study on implementation recommender system.

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import linear_kernel

# Sample movie dataset

data={'title':['Movie1','Movie2','Movie3','Movie4','Movie5'],

      'genres':['Action','Comedy','Drama','Action|Adventure','Comedy|Drama'],

      'keywords':['explosion, adventure, hero','laughter, funny, humor','emotions, love, tragedy',

                  'action, adventure, hero','laughter, love, drama']}}

movies_df=pd.DataFrame(data)

# Create a TF-IDF Vectorizer to convert text features into numerical vectors

tfidf_vectorizer=TfidfVectorizer(stop_words='english')

# Combine relevant text features into a single string

movies_df['combined_features']=movies_df['genres']+' '+movies_df['keywords']

# Apply TF-IDF Vectorization on the combined features

tfidf_matrix=tfidf_vectorizer.fit_transform(movies_df['combined_features'])

# Compute the cosine similarity between movies

cosine_sim=linear_kernel(tfidf_matrix,tfidf_matrix)

# Function to recommend movies based on user's input movie

def recommend_movies(movie_title,num_recommendations=3):

    movie_index=movies_df.index[movies_df['title']==movie_title].tolist()[0]

    similar_movies=list(enumerate(cosine_sim[movie_index]))

    # Sort similar movies by similarity score

    similar_movies=sorted(similar_movies,key=lambda x:x[1],reverse=True)
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

(AUTONOMOUS)

DEPARTMENT OF CSE(AI&ML)

Get the top N most similar movies (excluding the input movie itself)

```
top_similar_movies=similar_movies[1:num_recommendations+1]
```

```
recommended_movies=[movies_df['title'].iloc[movie[0]] for movie in top_similar_movies]
```

```
return recommended_movies
```

Example: Recommend movies similar to 'Movie1'

```
input_movie='Movie1'
```

```
recommended_movies=recommend_movies(input_movie,num_recommendations=3)
```

```
print(f'Recommended movies similar to '{input_movie}':')
```

```
for movie in recommended_movies:
```

```
    print(movie)
```

OUTPUT

```
Recommended movies similar to 'Movie1':
```

```
Movie4
```

```
Movie2
```

```
Movie3
```